Experiment No 2

Aim: Create a Blockchain using Python

Theory:
Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.

2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).

3. **Hashing** – SHA-256 algorithm ensures data immutability.

4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.

5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

1) What is a Blockchain?

A **Blockchain** is a **distributed and decentralized digital ledger** that stores data in the form of **blocks**.
Each block contains data, a timestamp, and the hash of the previous block, which links the blocks together in a chain.
Once data is added to the blockchain, it **cannot be easily altered**, ensuring **security, transparency, and immutability**.

2) Process of Mining

Mining is the process of **adding a new block** to the blockchain.

Steps involved in mining:

1. The miner collects transaction data.
2. A **nonce** is generated and combined with block data.
3. The miner computes a **hash using SHA-256**.
4. This process is repeated until the hash satisfies the **difficulty condition** (e.g., starts with leading zeros).
5. Once found, the block is added to the blockchain and broadcast to the network.

Mining ensures **security and consensus** in the blockchain.

3) How to Check the Validity of Blocks in a Blockchain

The validity of a blockchain is checked by verifying each block using the following conditions:

1. The **previous hash** stored in a block must match the **actual hash of the previous block**.
2. The **Proof-of-Work** must be valid (hash satisfies the difficulty condition).
3. Any change in block data changes the hash, making tampering easily detectable.

If all blocks satisfy these conditions, the blockchain is considered **valid and secure**.

Code:

```
# Module 1 - Create a Blockchain (Google Colab Compatible)
# Author: Aditi


import datetime
import hashlib
import json
from flask import Flask, jsonify
from pyngrok import ngrok

# PART 0: NGROK AUTHENTICATION
```

```python
ngrok.set_auth_token("38y0QMAnsudiiLDsyX5wExrSLnC_6xUsiVgNk5ShXdpaCcBbS")

# PART 1: BLOCKCHAIN STRUCTURE

class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_block(proof=1, previous_hash='0')  # Genesis block

    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash
        }
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    # PART 2: PROOF OF WORK

    def proof_of_work(self, previous_proof):
        new_proof = 1
        while True:
            hash_operation = hashlib.sha256(
                str(new_proof**2 - previous_proof**2).encode()
            ).hexdigest()
            if hash_operation[:4] == '0000':
                return new_proof
            new_proof += 1


    # PART 3: HASHING

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys=True).encode()
```

```
        return hashlib.sha256(encoded_block).hexdigest()

    # PART 4: BLOCKCHAIN VALIDATION

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        index = 1

        while index < len(chain):
            block = chain[index]

            if block['previous_hash'] != self.hash(previous_block):
                return False

            hash_operation = hashlib.sha256(
                                                str(block['proof']**2   -
previous_block['proof']**2).encode()
            ).hexdigest()

            if hash_operation[:4] != '0000':
                return False

            previous_block = block
            index += 1

        return True

# PART 5: FLASK APPLICATION

app = Flask(__name__)
blockchain = Blockchain()

@app.route('/mine_block', methods=['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    proof = blockchain.proof_of_work(previous_block['proof'])
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)

    return jsonify({
```

```
        'message': 'Block mined successfully',
        'block': block,
        'created_by': 'Aditi'
    }), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'valid': blockchain.is_chain_valid(blockchain.chain)
    }), 200


# PART 6: START NGROK + FLASK SERVER

public_url = ngrok.connect(5000)
print("Public URL:", public_url)

app.run(port=5000)
```

Output:

```
Public URL: NgrokTunnel: "https://marta-petticoated-hoarily.ngrok-free.dev" -> "http://localhost:5000"
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug:127.0.0.1 - - [30/Jan/2026 07:04:43] "GET /mine_block HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [30/Jan/2026 07:04:44] "GET /favicon.ico HTTP/1.1" 404 -
INFO:werkzeug:127.0.0.1 - - [30/Jan/2026 07:04:47] "GET /mine_block HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [30/Jan/2026 07:05:11] "GET /get_chain HTTP/1.1" 200 -
INFO:werkzeug:127.0.0.1 - - [30/Jan/2026 07:05:48] "GET /is_valid HTTP/1.1" 200 -
```

marta-petticoated-hoarily.ngrok-free.dev/get_chain

Pretty-print


```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-01-30 07:03:06.896176"
    },
    {
      "index": 2,
      "previous_hash": "2b716954ea01dd582b8bfcf7b07c404c96893774fbb33a6524766bdf55654159",
      "proof": 533,
      "timestamp": "2026-01-30 07:04:43.769887"
    },
    {
      "index": 3,
      "previous_hash": "0369132d230521b71f2c8b2d82bb5712094b0007c28e28e24e27c25114d5e8ca",
      "proof": 45293,
      "timestamp": "2026-01-30 07:04:47.805420"
    }
  ],
  "length": 3
}
```


marta-petticoated-hoarily.ngrok-free.dev/mine_block

Pretty-print


```
{
  "block": {
    "index": 4,
    "previous_hash": "526533e7222f2d4ccbd0e366a13eff61c3890786d3614009f2dc764ef5091520",
    "proof": 21391,
    "timestamp": "2026-01-30 07:14:06.189172"
  },
  "created_by": "Aditi",
  "message": "Block mined successfully"
}
```


marta-petticoated-hoarily.ngrok-free.dev/is_valid

Pretty-print


```
{
  "valid": true
}
```

Conclusion:

In this experiment, a basic blockchain was successfully implemented using Python and Flask. The system supports block mining, viewing the blockchain, and validating its integrity. The **Proof-of-Work mechanism** adds security by requiring computational effort, while **cryptographic hashing** ensures that data remains immutable. This implementation clearly demonstrates the fundamental principles of blockchain such as security, immutability, and consensus, and serves as a foundation for understanding more advanced blockchain applications.